

CAHIER DES CHARGES

Application mobile de forum communautaire

Projet Android Studio - Java - SQLite - Fonctionnement local/offline

Élément	Description
Nom du projet	CampusForum
Type de projet	Application mobile Android de forum communautaire
Langage imposé	Java
Base de données imposée	SQLite locale
Mode de fonctionnement	Local/offline, sans dépendance obligatoire à Internet
Public cible	Étudiants, enseignants, membres d'une communauté académique
Version du document	1.0
Date	01/06/2026

Ce document décrit les besoins, les fonctionnalités, les contraintes techniques, les données, les écrans et les critères de validation de l'application mobile CampusForum.

Sommaire

1. Présentation générale
2. Contexte et problématique
3. Objectifs du projet
4. Périmètre fonctionnel
5. Rôles et acteurs
6. Besoins fonctionnels
7. Besoins non fonctionnels
8. Architecture technique
9. Base de données SQLite
10. Écrans et navigation
11. Règles de gestion
12. Plan de réalisation
13. Critères de validation
14. Risques et solutions
15. Livrables attendus
16. Références techniques

1. Présentation générale

CampusForum est une application mobile Android permettant à des utilisateurs inscrits de participer à un forum communautaire local. L'application permet de créer des sujets de discussion, de consulter les sujets existants, de publier des réponses, de modifier ou supprimer ses propres contenus et de consulter un tableau de bord résumant l'activité.

Le projet est conçu pour fonctionner sans connexion Internet. Les données sont stockées localement dans une base SQLite embarquée dans l'application Android.

1.1 Finalité du projet

- Faciliter les échanges entre membres d'une même communauté, par exemple des étudiants d'une université.
- Permettre la gestion complète de discussions grâce aux opérations CRUD.
- Démontrer la maîtrise d'Android Studio, Java, SQLite, interfaces XML, RecyclerView et logique de navigation.
- Fournir un projet testable localement lors de la présentation, même en absence de connexion Internet.

2. Contexte et problématique

Dans un environnement universitaire, les étudiants ont souvent besoin de poser des questions, partager des ressources, demander de l'aide ou discuter autour de différents thèmes. Une application de forum communautaire répond à ce besoin en centralisant les échanges dans une interface mobile simple.

La contrainte principale du projet est l'utilisation de SQLite et du langage Java. Le système doit donc gérer localement les comptes, les publications, les réponses et les statistiques sans utiliser Firebase ni serveur distant.

3. Objectifs du projet

Objectif	Description
Authentification locale	Permettre à un utilisateur de créer un compte, se connecter, se déconnecter et récupérer son mot de passe via une méthode locale.
CRUD complet	Permettre la création, lecture, modification et suppression des sujets et réponses.
Dashboard	Afficher un résumé de l'activité : nombre de sujets, réponses, utilisateurs, catégories et dernières activités.
Stockage local	Persister toutes les données dans SQLite.
Interface claire	Proposer une interface simple, cohérente et adaptée à un étudiant débutant.
Présentation fiable	Garantir que l'application fonctionne localement le jour de la soutenance.

4. Périmètre fonctionnel

4.1 Fonctionnalités obligatoires

- Inscription utilisateur.
- Connexion utilisateur.
- Déconnexion.
- Récupération/réinitialisation locale du mot de passe.
- Création d'un sujet de discussion.
- Affichage de la liste des sujets.
- Affichage du détail d'un sujet.
- Modification et suppression d'un sujet par son auteur ou par un administrateur.
- Création, affichage, modification et suppression des réponses.
- Dashboard utilisateur et/ou administrateur.
- Gestion des catégories.

4.2 Fonctionnalités optionnelles

- Likes sur les sujets.
- Recherche par mot-clé.
- Filtrage par catégorie.
- Signalement de contenu.
- Profil utilisateur avec bio.
- Rôle administrateur.
- Mode sombre ou personnalisation simple de l'interface.

5. Acteurs et rôles

Acteur	Droits principaux
Visiteur	Accès aux écrans de connexion, inscription et récupération de mot de passe.
Utilisateur	Créer, lire, modifier et supprimer ses propres sujets et réponses. Consulter son dashboard.
Administrateur	Gérer les catégories, consulter les statistiques globales, supprimer les contenus inappropriés et visualiser les utilisateurs.

6. Besoins fonctionnels détaillés

ID	Fonctionnalité	Description	Priorité
F01	Inscription	Créer un compte avec nom, email, mot de passe et question/réponse de sécurité.	Haute
F02	Connexion	Authentifier un utilisateur à partir de son email et mot de passe.	Haute
F03	Déconnexion	Fermer la session locale de l'utilisateur.	Haute
F04	Mot de passe oublié	Réinitialiser localement le	Moyenne

		mot de passe après vérification de la réponse de sécurité.	
F05	Créer un sujet	Publier un sujet avec titre, contenu et catégorie.	Haute
F06	Lire les sujets	Afficher la liste des sujets récents avec auteur, catégorie et nombre de réponses.	Haute
F07	Modifier un sujet	Autoriser l'auteur à modifier son sujet.	Haute
F08	Supprimer un sujet	Autoriser l'auteur ou l'administrateur à supprimer un sujet.	Haute
F09	Répondre à un sujet	Ajouter une réponse à une discussion.	Haute
F10	Modifier/supprimer une réponse	Gérer les réponses de l'utilisateur connecté.	Haute
F11	Dashboard	Afficher les statistiques principales et activités récentes.	Haute
F12	Recherche	Rechercher un sujet par titre ou contenu.	Moyenne
F13	Gestion catégories	Créer et afficher les catégories du forum.	Moyenne

7. Besoins non fonctionnels

Besoin	Exigence
Disponibilité	L'application doit fonctionner sans Internet.
Performance	Les listes doivent rester fluides grâce à RecyclerView.
Maintenabilité	Le code doit être organisé en packages : ui, model, database, dao, repository, adapter, utils.
Sécurité minimale	Les mots de passe ne doivent pas être affichés en clair dans l'interface. Un hachage local est recommandé.
Ergonomie	Les écrans doivent être simples, lisibles et cohérents.
Compatibilité	Application Android développée dans Android Studio en Java.
Fiabilité	Les erreurs de saisie doivent être contrôlées : champs vides, email déjà utilisé, titre trop court, etc.

8. Architecture technique

L'architecture retenue est une architecture locale en couches, adaptée à Java, XML et SQLite.

Couche	Contenu	Rôle
UI	Activities, Fragments, XML Layouts,	Afficher les écrans et récupérer les

	RecyclerView Adapters	actions utilisateur.
Repository	AuthRepository, TopicRepository, ReplyRepository, DashboardRepository	Centraliser la logique d'accès aux données.
DAO local	UserDao, TopicDao, ReplyDao, CategoryDao	Exécuter les requêtes SQLite.
Database	DatabaseHelper extends SQLiteOpenHelper	Créer et mettre à jour la base SQLite.
Model	User, Topic, Reply, Category	Représenter les objets manipulés par l'application.
Utils	SessionManager, PasswordUtils, DateUtils	Gérer la session, la sécurité locale et les dates.

8.1 Structure recommandée du projet

```

com.example.campusforum
├── adapter
│   ├── TopicAdapter.java
│   ├── ReplyAdapter.java
│   └── CategoryAdapter.java
├── dao
│   ├── UserDao.java
│   ├── TopicDao.java
│   ├── ReplyDao.java
│   └── CategoryDao.java
├── database
│   └── DatabaseHelper.java
├── model
│   ├── User.java
│   ├── Topic.java
│   ├── Reply.java
│   └── Category.java
├── repository
│   ├── AuthRepository.java
│   ├── TopicRepository.java
│   ├── ReplyRepository.java
│   └── DashboardRepository.java
├── ui
│   ├── auth
│   ├── home
│   ├── topic
│   ├── dashboard
│   └── profile
└── utils
    ├── SessionManager.java
    ├── PasswordUtils.java
    └── DateUtils.java

```

9. Base de données SQLite

9.1 Tables principales

Table	Rôle
users	Stocker les comptes utilisateurs.
categories	Stocker les catégories de discussion.
topics	Stocker les sujets publiés.
replies	Stocker les réponses aux sujets.
likes	Optionnel : stocker les likes.
reports	Optionnel : stocker les signalements.

9.2 Schéma SQL proposé

```
CREATE TABLE users (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    username TEXT NOT NULL,  
    email TEXT NOT NULL UNIQUE,  
    password_hash TEXT NOT NULL,  
    security_question TEXT,  
    security_answer_hash TEXT,  
    role TEXT NOT NULL DEFAULT 'USER',  
    bio TEXT,  
    created_at TEXT NOT NULL,  
    is_active INTEGER NOT NULL DEFAULT 1  
);  
  
CREATE TABLE categories (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT NOT NULL UNIQUE,  
    description TEXT,  
    created_at TEXT NOT NULL  
);  
  
CREATE TABLE topics (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    title TEXT NOT NULL,  
    content TEXT NOT NULL,  
    author_id INTEGER NOT NULL,  
    category_id INTEGER NOT NULL,  
    created_at TEXT NOT NULL,  
    updated_at TEXT,  
    is_deleted INTEGER NOT NULL DEFAULT 0,  
    FOREIGN KEY(author_id) REFERENCES users(id),  
    FOREIGN KEY(category_id) REFERENCES categories(id)  
);  
  
CREATE TABLE replies (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    topic_id INTEGER NOT NULL,
```

```

    author_id INTEGER NOT NULL,
    content TEXT NOT NULL,
    created_at TEXT NOT NULL,
    updated_at TEXT,
    is_deleted INTEGER NOT NULL DEFAULT 0,
    FOREIGN KEY(topic_id) REFERENCES topics(id),
    FOREIGN KEY(author_id) REFERENCES users(id)
);

```

9.3 Relations

Relation	Cardinalité
Un utilisateur crée plusieurs sujets	users 1,N topics
Un utilisateur publie plusieurs réponses	users 1,N replies
Une catégorie contient plusieurs sujets	categories 1,N topics
Un sujet contient plusieurs réponses	topics 1,N replies

10. Écrans et navigation

Écran	Description
SplashActivity	Vérifie si une session locale existe et redirige vers LoginActivity ou MainActivity.
LoginActivity	Connexion avec email et mot de passe.
RegisterActivity	Création de compte.
ForgotPasswordActivity	Réinitialisation locale du mot de passe.
MainActivity / HomeFragment	Liste des sujets, recherche et accès aux catégories.
TopicDetailActivity	Affichage du sujet et de ses réponses.
CreateTopicActivity	Création d'un nouveau sujet.
EditTopicActivity	Modification d'un sujet existant.
DashboardActivity	Statistiques utilisateur ou administrateur.
ProfileActivity	Informations du compte et déconnexion.
AdminActivity	Optionnel : gestion des catégories et modération.

10.1 Parcours utilisateur

```

Lancement application
-> SplashActivity
    -> si session active : MainActivity
    -> sinon : LoginActivity
LoginActivity
-> RegisterActivity
-> ForgotPasswordActivity
-> MainActivity après connexion
MainActivity
-> CreateTopicActivity
-> TopicDetailActivity
-> DashboardActivity
-> ProfileActivity

```


TopicDetailActivity

- > ajouter réponse
- > modifier/supprimer sujet si auteur
- > modifier/supprimer réponse si auteur

11. Règles de gestion

Code	Règle
RG01	Un email ne peut être utilisé que par un seul compte.
RG02	Un utilisateur non connecté ne peut pas créer de sujet ni répondre.
RG03	Seul l'auteur d'un sujet peut le modifier ou le supprimer, sauf administrateur.
RG04	Seul l'auteur d'une réponse peut la modifier ou la supprimer, sauf administrateur.
RG05	La suppression peut être logique avec is_deleted = 1 afin de conserver l'historique.
RG06	Un sujet doit avoir un titre, un contenu et une catégorie.
RG07	Une réponse ne peut pas être vide.
RG08	Le dashboard doit être calculé à partir des données SQLite.
RG09	La récupération de mot de passe se fait localement via question/réponse de sécurité ou réinitialisation administrateur.
RG10	L'application doit être présentable sans connexion Internet.

12. Plan de réalisation

Phase	Travail à réaliser	Résultat attendu
1	Créer le projet Android Studio Java, définir le package, préparer Git ou sauvegarde.	Projet initial propre.
2	Concevoir les écrans XML de base : login, register, home, detail, dashboard.	Maquettes fonctionnelles.
3	Créer DatabaseHelper et les tables SQLite.	Base locale opérationnelle.
4	Créer les classes Model et DAO.	Accès aux données structuré.
5	Implémenter l'authentification locale.	Inscription, connexion, session, déconnexion.
6	Implémenter le CRUD des catégories et sujets.	Liste, création, détail, modification, suppression.
7	Implémenter le CRUD des réponses.	Réponses fonctionnelles dans chaque sujet.
8	Développer le dashboard.	Statistiques utilisateur/admin.
9	Ajouter recherche, filtres et	Application plus agréable.

	améliorations UI.	
10	Tester, corriger, préparer démonstration et rapport.	Version finale présentable.

13. Critères de validation

- L'application s'installe et se lance correctement sur émulateur ou téléphone Android.
- Un utilisateur peut créer un compte local.
- Un utilisateur peut se connecter et se déconnecter.
- La session est conservée localement si nécessaire.
- Les données restent disponibles après fermeture et réouverture de l'application.
- Les opérations CRUD sur les sujets fonctionnent.
- Les opérations CRUD sur les réponses fonctionnent.
- Le dashboard affiche des statistiques correctes.
- L'interface est lisible et les erreurs de saisie sont gérées.
- La démonstration fonctionne sans Internet.

14. Risques et solutions

Risque	Impact	Solution
Base SQLite mal conçue	Difficultés pour les requêtes et le dashboard.	Définir le schéma avant de coder et tester chaque table.
Trop de fonctionnalités	Projet non terminé.	Commencer par le MVP : Auth + Topics + Replies + Dashboard.
Mot de passe oublié sans Internet	Impossible d'envoyer un email.	Utiliser une question de sécurité ou un reset admin local.
RecyclerView difficile	Liste non fonctionnelle.	Commencer par afficher des données statiques puis connecter SQLite.
Bugs de session	Mauvaise redirection.	Utiliser SessionManager avec SharedPreferences ou table session.
Suppression définitive risquée	Perte de données.	Préférer la suppression logique avec is_deleted.
Présentation sans données	Démonstration pauvre.	Insérer des catégories et données de test au premier lancement.

15. Livrables attendus

- Code source Android Studio complet.
- APK ou démonstration sur émulateur/téléphone.
- Base SQLite créée automatiquement au lancement.
- Cahier des charges.
- Rapport de projet ou documentation utilisateur.
- Captures d'écran des principaux écrans.
- Jeu de données de démonstration.

16. Références techniques

Les références suivantes peuvent être utilisées pour justifier les choix techniques dans le rapport :

- Android Developers - SQLite : <https://developer.android.com/training/data-storage/sqlite>
- Android Developers - SQLiteOpenHelper :
<https://developer.android.com/reference/android/database/sqlite/SQLiteOpenHelper>
- Android Developers - SQLiteDatabase :
<https://developer.android.com/reference/android/database/sqlite/SQLiteDatabase>
- Android Developers - Guide to app architecture :
<https://developer.android.com/topic/architecture>
- Android Developers - RecyclerView :
<https://developer.android.com/develop/ui/views/layout/recyclerview>
- Android Developers - SharedPreferences :
<https://developer.android.com/training/data-storage/shared-preferences>